برداشت از کتاب Python for Data Analysis نوشته Wes McKinney

فصل چهارم: نامپای (NumPy)

- نامپای سریعتر از کد پایتون
  - ۱۰ تا ۱۰۰ برابر سریعتر
  - استفاده از حافظه بسیار کمتر

مثال صفحه ۸۵:

```
my_arr = np.arange(1_000_000)
my_list = list(range(1_000_000))
%timeit my_arr2 = my_arr * 2
%timeit my_list2 = [x * 2 for x in my_list]
```

۱.۴. شی آرایه n بعدی(N-dimensional array object یا ndarray object)

- یکی از ویژگی های کلیدی NumPy، شی آرایه N بعدی آن یا ndarray است که یک ظرف سریع و انعطاف پذیر برای مجموعه داده های بزرگ در پایتون است. آرایه ها شما را قادر می‌سازند تا عملیات ریاضی را بر روی کل بلوک های داده با استفاده از نحو مشابه با، عملیات معادل بین عناصر اسکالر، انجام دهید. مثال صفحه ۸۵

■ **ایجاد آرایه یک بعدی، دوبعدی و بیشتر**

- استفاده از تابع array
- numpy.zeros
- numpy.ones
- numpy.empty: یک آرایه را بدون مقداردهی اولیه مقادیر آن به مقدار خاصی ایجاد می‌کند.
  - نکته: برای ایجاد آرایه‌های دوبعدی و بیشتر با استفاده از سه متد بالا یک تاپل را به عنوان اندازه ارسال کنید.
- numpy.arange: مانند تابع range پایتون است.
  - متد reshape

نکته: استفاده از خاصیت‌های shape و dtype وndim برای آرایه، مانند data.shape

■ **انواع داده‌ای برای ndarray ها (dtype): صفحه ۸۹ و ۹۰**

- تبدیل انواع داده‌ای به یکدیگر با متد astype
  - نکته۱: همچنین می‌توان از خاصیت dtype آرایه دیگری استفاده کرد.
  - نکته۲: رشته‌های کد نوع کوتاهی وجود دارد(ستون Type code در جدول) که می‌توانید از آنها برای ارجاع به dtype مورد نظر استفاده کرد.

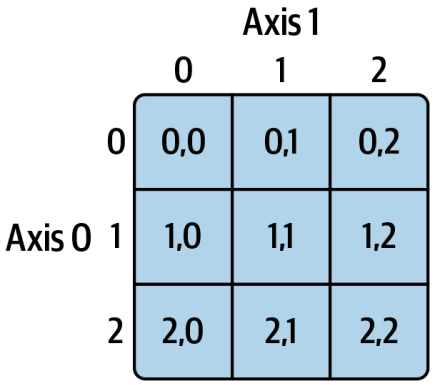
- نکته۳: فراخوانی astype همیشه یک آرایه جدید (یک کپی از داده ها) ایجاد می‌کند، حتی اگر نوع داده جدید با نوع داده قدیمی یکسان باشد.

■ محاسبات با آرایه‌های NumPy

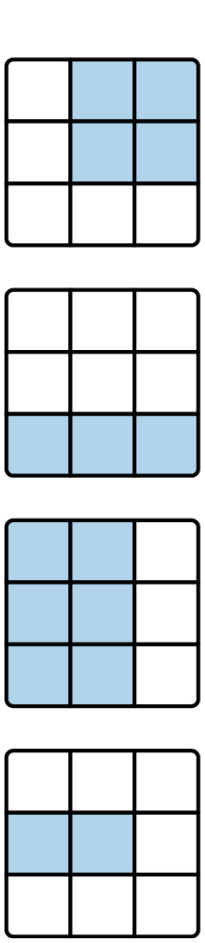
- عملیات برداری(*vectorization*)
  - نکته: هر عملیات حسابی بین آرایه‌های با اندازه مساوی(equal-size arrays)، عمل element-wise را اعمال می‌کند.
- انتشار عملیات حسابی با عدد اسکالر
- آرایه‌های بولین: حاصل مقایسه(با عملگرهای مقایسه‌ای) بین آرایه‌های هم اندازه، آرایه‌های بولین خواهد بود.

■ Basic Indexing and Slicing

- انتخاب زیرمجموعه‌ای از داده‌ها یا انتخاب عناصر خاص برای آرایه‌های تک بعدی
  - نکته۱: اولین وجه تمایز مهم آرایه‌های نامپای از لیست‌های داخلی پایتون، این است که برش‌های آرایه Viewهایی روی آرایه اصلی هستند. این بدان معنی است که داده‌ها کپی نمی‌شوند و هرگونه تغییر در View در آرایه اصلی منعکس می‌شود. مثال صفحه ۹۳
  - نکته۲: اگر می‌خواهید یک copy، از یک برش ndarray به جای view داشته باشید، باید صریحاً آرایه را کپی کنید: arr[5:8].copy() . همانطور که خواهید دید، pandas نیز به این روش کار می‌کنند.
- انتخاب زیرمجموعه‌ای از داده‌ها یا انتخاب عناصر خاص برای آرایه‌های دو بعدی
  - indexing elements in a NumPy array

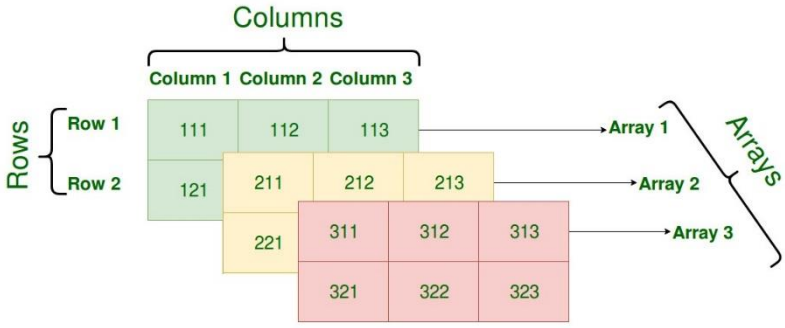


- Two-dimensional array slicing



Expression	Shape
arr[:2,1:]	(2,2)
arr[2]	(3,)
arr[2, :]	(3,)
arr[2:, :]	(1,3)
arr[:, :2]	(3,2)
arr[1, :2]	(2,)
arr[1:2, :2]	(1,2)

- انتخاب زیرمجموعه‌ای از داده‌ها یا انتخاب عناصر خاص برای آرایه‌های سه بعدی



■ Boolean Indexing

- آرایه بولین، آرایه‌ای است که مقادیر آن True یا False است. آرایه بولین را می‌توان به عنوان indexing یک آرایه استفاده کرد. مثال صفحه ۹۸: همچنین استفاده از عملگرهای =! ، ~ ، | ، &
  - نکته۱: کلمات کلیدی پایتون and و or با آرایه‌های بولین کار نمی‌کنند، بلکه باید از & و | استفاده کرد.

- نکته۲: انتخاب داده‌ها از یک آرایه با Boolean indexing و تخصیص نتیجه به یک متغیر جدید همیشه یک کپی از داده‌ها ایجاد می‌کند، حتی اگر آرایه برگشتی بدون تغییر باشد.

- تغییر مقادیر آرایه با شروط مورد نظر، توسط آرایه‌های بولین
  - اعمال شرط روی تک تک مقادیر
  - اعمال شرط روی سطرها یا ستون‌های خاص

مثال صفحه ۱۰۰

■ Fancy Indexing

- اصطلاحی است که توسط NumPy برای توصیف indexing با استفاده از آرایه‌های عدد صحیح به کار گرفته شده است. مثال صفحه ۱۰۰
- نتیجه Fancy indexing با چند آرایه های عدد صحیح به تعداد محورها، همیشه آرایه‌ای یک بعدی است. مثال صفحه ۱۰۱

■ جابجایی آرایه ها و مبادله محورها(Transposing Arrays and Swapping Axes)

- جابجایی(Transposing) شکل خاصی از تغییر شکل است که یک View از داده‌های زیرین را بدون کپی کردن چیزی بر می‌گرداند. استفاده از خاصیت T . مثال صفحه ۱۰۲
- ضرب ماتریس‌ها با numpy.dot و یا عملگر @
- به جای خاصیت T، می‌توان از متد swapaxes که یک حالت کلی‌تر است، استفاده کرد.

۲.۴. تولید اعداد شبه تصادفی(Pseudorandom Number Generation)

- ماژول numpy.random بسیار سریع‌تر از ماژول random داخلی پایتون است. مثال صفحه ۱۰۴
- با استفاده از تابع numpy.random.standard_normal می‌توان یک آرایه با توزیع نرمال استاندارد ایجاد کرد:

```
samples = np.random.standard_normal(size=(4, 4))
```

- تابع numpy.random.standard_normal از مولد اعداد تصادفی پیش‌فرض ماژول numpy.random استفاده می‌کند، اما می‌توان کد را به گونه‌ای نوشت که از یک مولد صریح پیکربندی شود، مانند:

```
rng = np.random.default_rng(seed=12345)
data = rng.standard_normal((2, 3))
```

  - نکته: اگر پارامتر seed را مشخص و مقداردهی کنید، فقط یکبار اعداد تصادفی تولید می‌شود و پس از هر بار اجرای کد اعداد تصادفی جدید تولید نمی‌شوند. توجه کنید rng یک مولد است.
  - متد standard_normal اعداد تصادفی توزیع نرمال با میانگین صفر و انحراف معیار یک را تولید می‌کند.

- برای تولید اعداد تصادفی صحیح می‌توان از متد integers مربوط به شی مولد به صورت زیر استفاده کرد:

```
rng = np.random.default_rng(seed=12345)
rng.integers(low=5, high=20, size=(2, 4))
```

۴.۳. توابع عمومی(Universal Functions: Fast Element-Wise Array Functions)

- یک تابع عمومی یا ufunc تابعی است که عملیات element-wise روی داده‌ها در ndarrayها انجام می‌دهد. مانند numpy.sqrt یا numpy.exp. اینها به عنوان unary ufuncs نامیده می‌شوند. برخی دیگر، مانند numpy.add یا numpy.maximum، دو آرایه (بنابراین، binary ufuncs) می‌گیرند و یک آرایه را به عنوان نتیجه برمی‌گردانند. مثال صفحه ۱۰۵
- معمول نیست که یک ufunc چند آرایه را برگرداند، اما تابع numpy.modf یک آرایه مقادیر اعشاری می‌گیرد و دو آرایه، یکی شامل بخش صحیح و دیگری شامل بخش اعشاری مقادیر است. مثال صفحه ۱۰۶
- Ufuncها یک آرگومان خروجی اختیاری را می‌پذیرند که به آنها اجازه می‌دهد تا نتایج خود را به جای ایجاد آرایه جدید به یک آرایه موجود اختصاص دهند. مثال صفحه ۱۰۶
- برخی unary universal functions و binary universal functions در جداول صفحه ۱۰۷ آمده‌اند.

۴.۴. برنامه نویسی آرایه‌گرا با آرایه‌ها(Array-Oriented Programming with Arrays)

- استفاده از آرایه‌های NumPy شما را قادر می‌سازد تا انواع مختلفی از وظایف پردازش داده را به صورت مختصر بیان کنید که در غیر این صورت ممکن است به نوشتن حلقه‌ها(loops) نیاز داشته باشید. این عمل جایگزینی حلقه‌های صریح با بیان آرایه‌ای، توسط برخی افراد به عنوان برداری(*vectorization*) نامیده می‌شود. به طور کلی، عملیات آرایه برداری شده معمولاً به طور قابل توجهی سریعتر از معادل‌های پایتون خالص خود هستند و بیشترین تأثیر را در هر نوع محاسبات عددی دارند.
- اصطلاح برداری(*vectorization*) برای توصیف برخی دیگر از مفاهیم علوم کامپیوتر استفاده می‌شود، اما در این کتاب، برای توصیف عملیات روی کل داده‌های آرایه به طور همزمان استفاده می‌شود، نه اینکه با استفاده از یک حلقه پایتون مقدار به مقدار این کار را انجام شود.
- مثال: xs و ys دو آرایه دوبعدی هستند:

```
z = np.sqrt(xs ** 2 + ys ** 2)
```

■ بیان منطق شرطی به عنوان عملیات آرایه(Expressing Conditional Logic as Array Operations)

- تابع numpy.where یک نسخه برداری از عبارت سه گانه x if condition else y می‌باشد. مثال:

```
xarr = np.array([1.1, 1.2, 1.3, 1.4, 1.5])
yarr = np.array([2.1, 2.2, 2.3, 2.4, 2.5])
cond = np.array([True, False, True, True, False])
```

می‌خواهیم اگر شرط درست بود انتخاب مقدار از آرایه xarr و در غیر این صورت از آرایه yarr مقدار گرفته شود.

الف) با استفاده از حلقه for :

```
result = [(x if c else y) for x, y, c in zip(xarr, yarr, cond)]
```

ب) با روش برداری:

```
result = np.where(cond, xarr, yarr)
```

حل با روش "الف" مشکلات متعددی دارد. اولاً، برای آرایه های بزرگ خیلی سریع نخواهد بود (زیرا تمام کارها در کد پایتون تفسیر شده انجام می شود). دوم اینکه با آرایه های چند بعدی کار نمی کند. در حالیکه با numpy.where می‌توانید این کار را با یک فراخوانی تک تابع انجام دهید.

- آرگومان دوم و سوم numpy.where نیازی نیست حتماً آرایه باشند، بلکه یکی یا هر دوی آنها می‌توانند عدد اسکالر باشند. مثال: صفحه ۱۱۰ و ۱۱۱

■ متدهای ریاضی و آماری(Mathematical and Statistical Methods)

- استفاده از توابع sum, mean, std
  - با فراخوانی متد نمونه آرایه(array instance method)، مانند: arr.mean()
  - استفاده از تابع سطح بالای نامپای(top-level NumPy function) مانند: np.mean(arr)
- توابعی مانند میانگین و مجموع یک آرگومان محور(axis) اختیاری می گیرند که آمار را بر روی محور داده شده محاسبه می کند و در نتیجه آرایه ای با یک بعد کمتر ایجاد می شود:

```
arr.mean(axis=1)
arr.sum(axis=0)
```

arr.mean(axis=1) به معنی compute mean across the columns یعنی محاسبه میانگین در سراسر ستون‌ها(در واقع محاسبه سطر به سطر انجام می‌شود:خ)

arr.sum(axis=0) به معنی compute sum down the rows یعنی محاسبه مجموع با گذر از سطرها به طرف پایین (در واقع محاسبه ستون به ستون انجام می‌شود:خ)

- استفاده از متدهای cumsum, cumprod مثال:

```
arr = np.array([0, 1, 2, 3, 4, 5, 6, 7])
In [200]: arr.cumsum()
Out[200]: array([ 0, 1, 3, 6, 10, 15, 21, 28])
```

- متدهای پایه آماری آرایه (Basic array statistical methods) در جدول صفحه ۱۱۳ آمده‌اند.

■ متدهای برای آرایه‌های بولین(Methods for Boolean Arrays)

- در این متدها، مقادیر بولین اگر False باشند به صفر و اگر True باشند به یک ترجمه می‌شوند. بنابراین، sum اغلب به عنوان وسیله‌ای برای شمارش مقادیر True در یک آرایه بولی استفاده می‌شود:

```
arr = rng.standard_normal(100)
(arr > 0).sum() # Number of positive values
(arr <= 0).sum() # Number of non-positive values
```

- دو متد any, all : any تست می کند که آیا یک یا چند مقدار در یک آرایه True است، در حالی که all بررسی می‌کند که آیا همه مقادیر True هستند یا خیر:

```
bools = np.array([False, False, True, False])
bools.any()  # True    ;    bools.all() # False
```

- نکته: این متدها همچنین با آرایه‌های غیر بولین کار می‌کنند، به گونه‌ای که عناصر غیر صفر به عنوان True در نظر گرفته می‌شوند.

■ مرتب سازی(Sorting)

- مانند نوع لیست داخلی پایتون، آرایه‌های NumPy را می‌توان با متد sort در جای خود(in place) مرتب کرد:

```
arr.sort()
```

- در آرایه‌های چندبعدی، می‌توان مرتب سازی را حول یک محور خاص به صورت in place انجام داد:

مرتب سازی در هر ستون arr.sort(axis=0)

مرتب سازی در هر سطر arr.sort(axis=1)

- متد سطح بالا numpy.sort یک کپی مرتب شده از یک آرایه (مانند تابع sorted داخلی پایتون) را به جای اینکه آرایه را در جای خود تغییر دهد، برمی‌گرداند. به عنوان مثال:

```
arr2 = np.array([5, -10, 7, 1, 0, -3])
sorted_arr2 = np.sort(arr2)
```

■ منحصر به فرد و سایر منطق مجموعه(Unique and Other Set Logic)

- NumPy دارای چند عملیات مجموعه پایه برای آرایه‌های یک بعدی است. یکی از رایج ترین موارد استفاده numpy.unique است که مقادیر منحصر به فرد مرتب شده را در یک آرایه بر می‌گرداند:

```
names = np.array(["Bob", "Will", "Joe", "Bob", "Will", "Joe", "Joe"])
np.unique(names)
Out[225]: array(['Bob', 'Joe', 'Will'], dtype='<U4')
or
ints = np.array([3, 3, 3, 2, 2, 1, 1, 4, 4])
np.unique(ints)
Out[227]: array([1, 2, 3, 4])
```

numpy.unique را با صرفاً با کد پایتون مقایسه کنید:

```
sorted(set(names))
Out[228]: ['Bob', 'Joe', 'Will']
```

توجه کنید که در بسیاری از موارد، نسخه NumPy سریعتر است و یک آرایه NumPy را به جای یک لیست Python برمی‌گرداند.

- تابع numpy.in1d، عضویت مقادیر یک آرایه در آرایه دیگر آزمایش می‌کند و یک آرایه بولین را بر می‌گرداند:

```
values = np.array([6, 0, 0, 3, 2, 5, 6])
np.in1d(values, [2, 3, 6])
Out[230]: array([ True, False, False, True, True, False, True])
```

- لیستی از عملیات‌های مجموعه‌ای آرایه در نامپای در جدول صفحه ۱۱۵ و ۱۱۶ آمده است.